Reference Sheet

Not everything on this reference sheet may be needed.

String Class API

```
public class String {
    /** Returns the character at the specified index (position) */
    public char charAt(int index) { ... }

    /** Converts this string to a new character array. */
    public char[] toCharArray() { ... }
}
```

Random Class API

```
public class Random {
    /** Creates a new random number generator. */
    public Random() { ... }

    /** Creates a new random number generator using a seed. If two instances of Random are created
    with the same seed, they will generate and return identical sequences of numbers. Runs in Θ(1)
    time. */
   public Random(long seed) { ... }

    /** Returns the next pseudorandom int from this random number generator's sequence. Runs in Θ(1)
    time. */
   public int nextInt() { ... }

    /** Returns a pseudorandom int value between 0 (inclusive) and the specified value (exclusive),
    drawn from this random number generator's sequence. Runs in Θ(1) time. */
   public int nextInt(int bound) { ... }
}
```

Queue Class

```
/** Represents a Queue in Java. This is not part of the Java Standard Library. */
public class Queue<E> {
   /** Create a queue. Runs in Θ(1) time. */
   public Queue() { ... }

   /** Create a queue, then inserts all elements of lst one at a time into the queue. Runs in Θ(N)
    time. */
   public Queue(List<E> lst) { ... }

   /** Inserts the specified element into this queue, returning true on success. Runs in Θ(1) time.
    */
   public boolean add(E e) { ... }
```

```
   /** Retrieves, but does not remove, the element at the head of the queue, or returns null if this
    queue is empty. Runs in Θ(1) time.*/
   public E peek() { ... }

   /** Retrieves and removes the element at the head of the queue. Runs in Θ(1) time.*/
   public E remove() { ... }

   /** Returns the size of the queue. Runs in Θ(1) time.*/
   public int size() { ... }
}
```

Stack Class

```
public class Stack<E> {
   /** Create a Stack. Runs in Θ(1) time. */
   public Stack() { ... }

   /** Create a Stack, then inserts all elements of lst one at a time into the stack. Runs in Θ(N)
    time. */
   public Stack(List<E> lst) { ... }

   /** Inserts the specified element into this stack returning true on success. Runs in Θ(1) time. */
   public boolean add(E e) { ... }

   /** Retrieves, but does not remove, the element at the top of the stack, or returns null if this
    stack is empty. Runs in Θ(1) time. */
   public E peek() { ... }

   /** Retrieves and removes the element at the top of the stack. Runs in Θ(1) time. */
   public E remove() { ... }

   /** Returns the size of the stack. Runs in Θ(1) time. */
   public int size() { ... }
}
```